Task 15:

Implement Service Registry and Discovery using a tool such as Eureka. Register multiple microservices and enable dynamic service lookup instead of hard-coded service URLs.

Aim

To implement **Service Registry and Discovery** using **Eureka Server**, register multiple microservices, and enable **dynamic service lookup** instead of hard-coded URLs.

Algorithm

1. Create a **Eureka Server** project.
2. Add dependency: spring-cloud-starter-netflix-eureka-server.
3. Enable Eureka Server using @EnableEurekaServer.
4. Configure server properties.
5. Run Eureka Server.
6. Modify microservices (Product & Order):
    - Add Eureka Client dependency.
    - Enable @EnableDiscoveryClient.
    - Configure Eureka server URL.
7. Register microservices with Eureka.
8. Start all services.
9. Verify registration on Eureka Dashboard.

Architecture Overview

- **Eureka Server** → Service registry
- **Product Service** → Registers itself
- **Order Service** → Registers itself

Code Implementation

1. Eureka Server

pom.xml Dependency

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

Main Class

```
package com.eurekaserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {

   public static void main(String[] args) {
      SpringApplication.run(EurekaServerApplication.class, args);
   }
}
```

application.properties

```
server.port=8761

eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

2. Product Service (Eureka Client)

Add Dependency

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Main Class Update

```
@EnableDiscoveryClient
@SpringBootApplication
public class ProductServiceApplication {
   public static void main(String[] args) {
      SpringApplication.run(ProductServiceApplication.class, args);
   }
}
```

application.properties

```
spring.application.name=PRODUCT-SERVICE
server.port=8081

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

- **3. Order Service (Eureka Client)**

Main Class

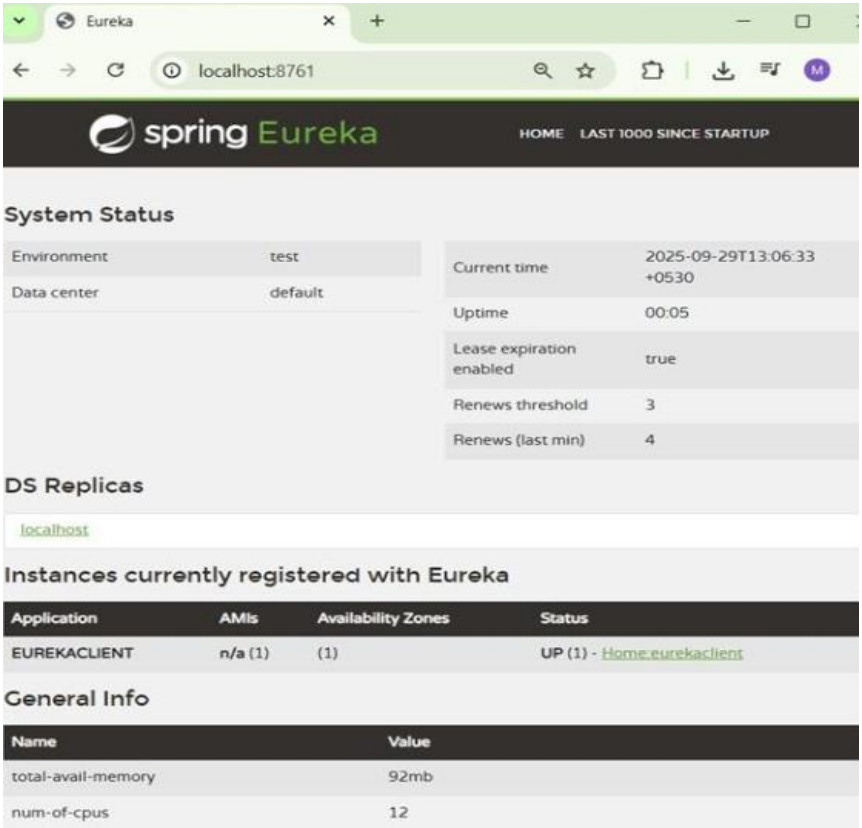
```
@EnableDiscoveryClient
@SpringBootApplication
public class OrderServiceApplication {
   public static void main(String[] args) {
      SpringApplication.run(OrderServiceApplication.class, args);
   }
}
```

application.properties

```
spring.application.name=ORDER-SERVICE
server.port=8082

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

Output:



Result

Successfully implemented **Service Registry using Eureka** Microservices dynamically registered No need for hardcoded URLs Enabled **service discovery and scalability**